
1. Task 1 – NumPy Introduction & Arrays

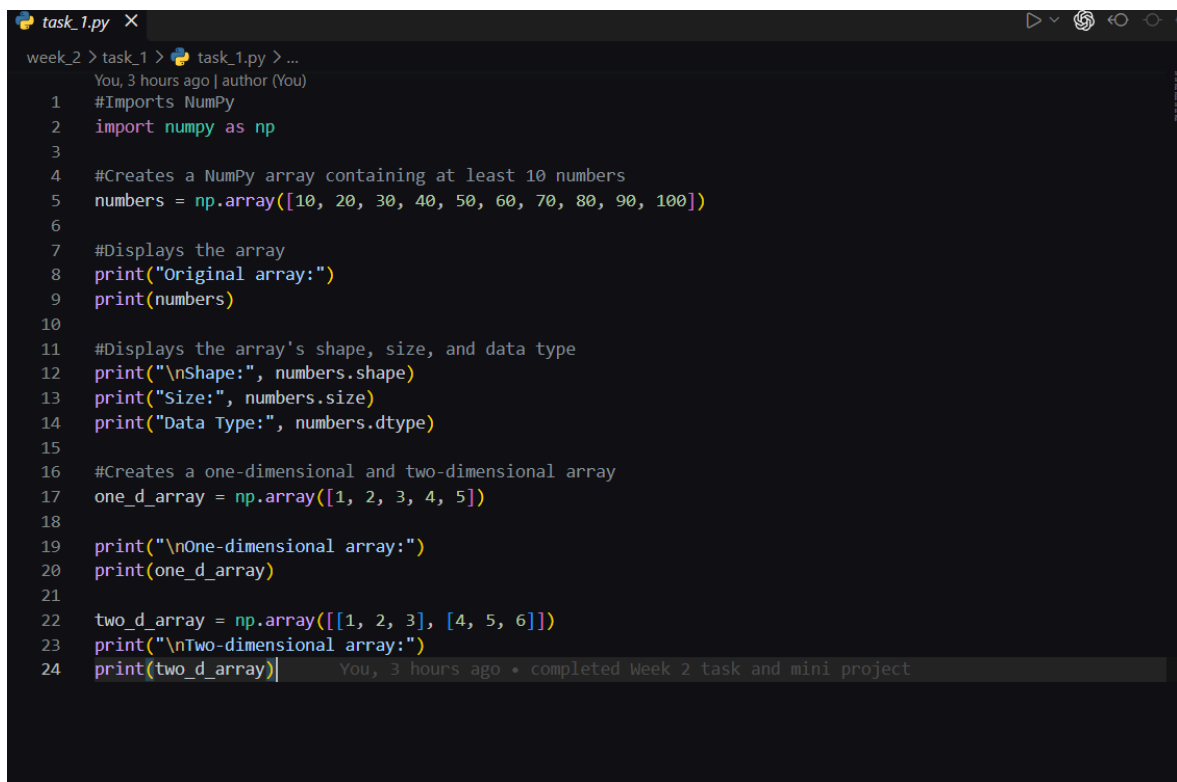
Objective

To understand the basics of NumPy arrays and their properties.

Brief Explanation

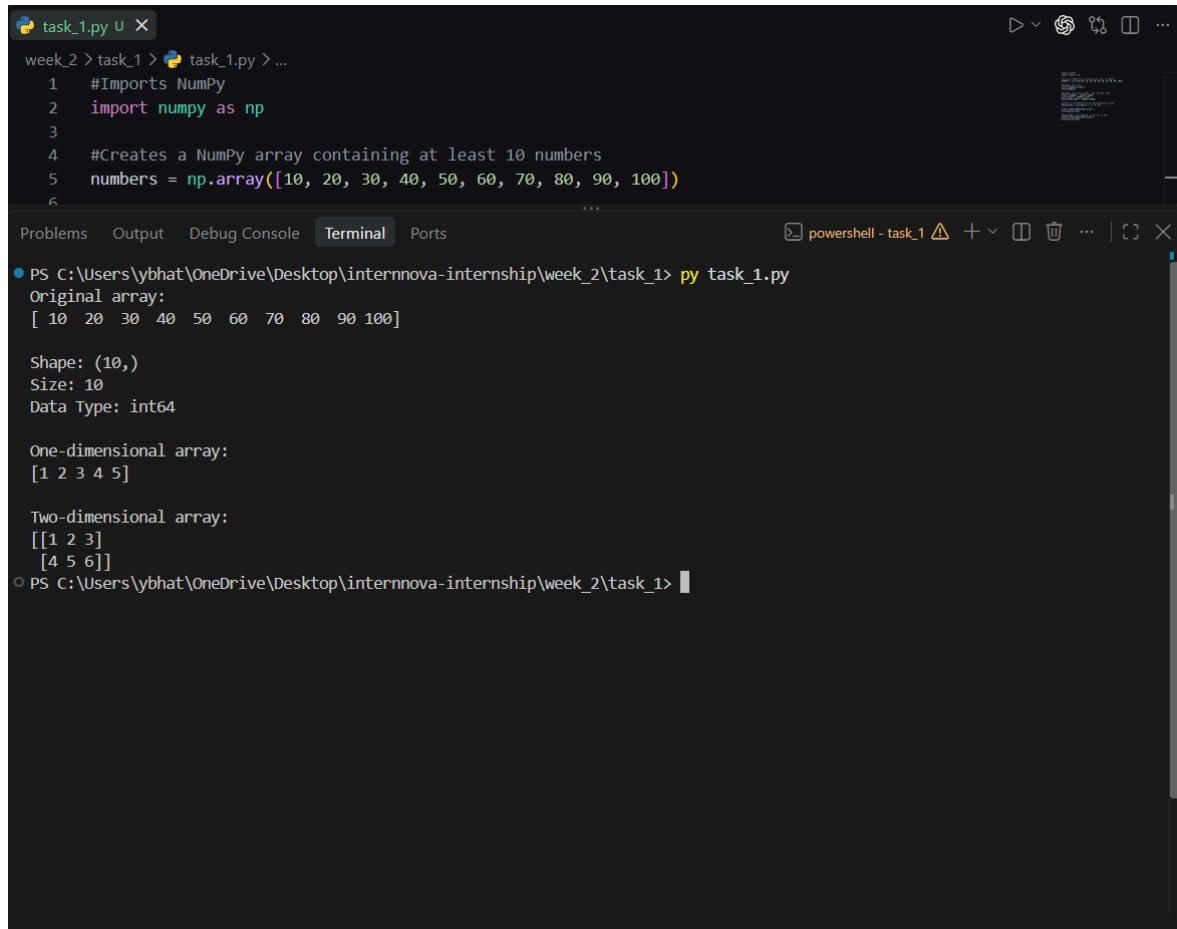
NumPy was used to create one-dimensional and two-dimensional arrays. The array values, shape, size, and data type were displayed.

Python Source Code



```
task_1.py X
week_2 > task_1 > task_1.py > ...
You, 3 hours ago | author (You)
1  #Imports NumPy
2  import numpy as np
3
4  #Creates a NumPy array containing at least 10 numbers
5  numbers = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
6
7  #Displays the array
8  print("Original array:")
9  print(numbers)
10
11 #Displays the array's shape, size, and data type
12 print("\nShape:", numbers.shape)
13 print("Size:", numbers.size)
14 print("Data Type:", numbers.dtype)
15
16 #Creates a one-dimensional and two-dimensional array
17 one_d_array = np.array([1, 2, 3, 4, 5])
18
19 print("\nOne-dimensional array:")
20 print(one_d_array)
21
22 two_d_array = np.array([[1, 2, 3], [4, 5, 6]])
23 print("\nTwo-dimensional array:")
24 print(two_d_array)
You, 3 hours ago • completed Week 2 task and mini project
```

Screenshot of Program Output



```
task_1.py U X
week_2 > task_1 > task_1.py > ...
1 #Imports NumPy
2 import numpy as np
3
4 #Creates a NumPy array containing at least 10 numbers
5 numbers = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])
6

Problems Output Debug Console Terminal Ports
powershell - task_1
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_1> py task_1.py
Original array:
[ 10  20  30  40  50  60  70  80  90 100]

Shape: (10,)
Size: 10
Data Type: int64

One-dimensional array:
[1 2 3 4 5]

Two-dimensional array:
[[1 2 3]
 [4 5 6]]
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_1>
```

2. Task 2 – NumPy Indexing, Slicing & Reshaping

Objective

To understand how to access, slice, and reshape NumPy arrays.

Brief Explanation

Array indexing and slicing were used to access specific elements and portions of an array. Two-dimensional arrays were used to access rows and columns, and arrays were reshaped into different dimensions.

Python Source Code

```
import numpy as np
```

```
numbers = np.array([10, 20, 30, 40, 50, 60, 70, 80, 90, 100])

print("Original 1D array:")

print(numbers)

#Access specific elements using indexing

print("\n Indexing:")

print("Element at index 0:", numbers[0])

print("Element at index 3:", numbers[3])

print("Last element:", numbers[-1])

#Extract a portion of the array using slicing

print("\nSlicing:")

print("Elements from index 2 to 5:", numbers[2:6])

print("First five elements:", numbers[:5])

print("Last three elements:", numbers[-3:])

print("Every second element:", numbers[::2])

print("Reversed array:", numbers[::-1])

#Create a two-dimensional array

two_d_array = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

print("\nTwo-dimensional array:")

print(two_d_array)
```

```
#Access specific rows and columns

print("\nAccessing rows and columns:")

print("First row:", two_d_array[0, :])

print("Second column:", two_d_array[:, 1])

print("Element at row 2, column 3:", two_d_array[1, 2])


#Reshape an array into different dimensions

reshape_numbers = np.arange(1, 13)

print("\nOriginal 1D array for reshaping:")

print(reshape_numbers)


#Reshape into a 3x4 array

reshaped_3x4 = reshape_numbers.reshape(3, 4)

print("\nReshaped into 3x4 array:")

print(reshaped_3x4)


#Reshape into a 4x3 array

reshaped_4x3 = reshape_numbers.reshape(4, 3)

print("\nReshaped into 4x3 array:")

print(reshaped_4x3)

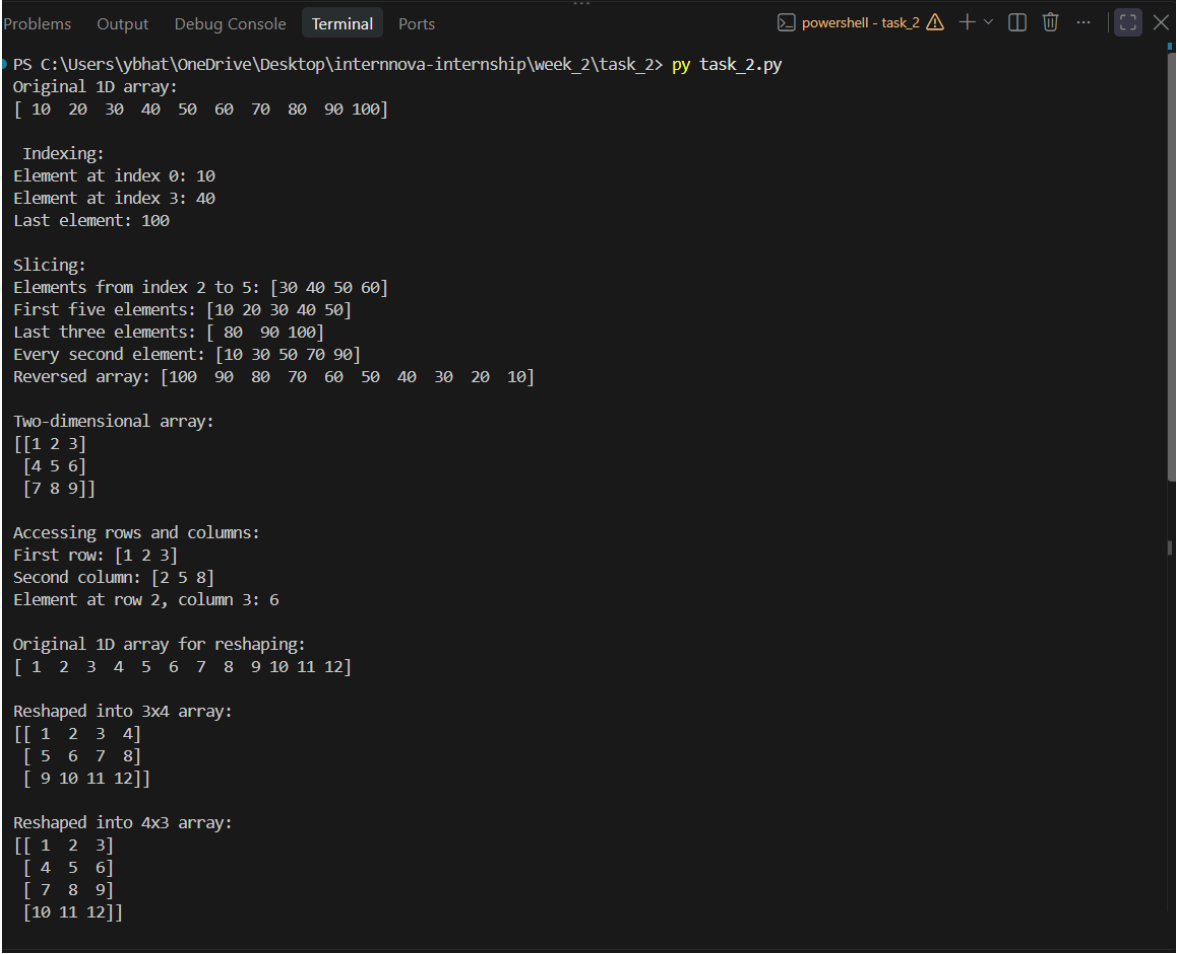

#Reshape into a 2x2x3 array
```

```
reshaped_2x2x3 = reshape_numbers.reshape(2, 2, 3)

print("\nReshaped into 3 Dimensions (2x2x3) array:")

print(reshaped_2x2x3)
```

Screenshot of Program Output



```
Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_2> py task_2.py
Original 1D array:
[ 10  20  30  40  50  60  70  80  90 100]

Indexing:
Element at index 0: 10
Element at index 3: 40
Last element: 100

Slicing:
Elements from index 2 to 5: [30 40 50 60]
First five elements: [10 20 30 40 50]
Last three elements: [ 80  90 100]
Every second element: [10 30 50 70 90]
Reversed array: [100  90  80  70  60  50  40  30  20  10]

Two-dimensional array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Accessing rows and columns:
First row: [1 2 3]
Second column: [2 5 8]
Element at row 2, column 3: 6

Original 1D array for reshaping:
[ 1  2  3  4  5  6  7  8  9 10 11 12]

Reshaped into 3x4 array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Reshaped into 4x3 array:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_2> py task_2.py

Original 1D array for reshaping:
[ 1  2  3  4  5  6  7  8  9 10 11 12]

Reshaped into 3x4 array:
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]

Reshaped into 4x3 array:
[[ 1  2  3]
 [ 4  5  6]
 [ 7  8  9]
 [10 11 12]]

Reshaped into 3 Dimensions (2x2x3) array:
[[[ 1  2  3]
   [ 4  5  6]]
 [[ 7  8  9]
   [10 11 12]]]
PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_2> 
```

3. Task 3 – NumPy Mathematical & Statistical Operations

Objective

To perform mathematical and statistical operations using NumPy.

Brief Explanation

NumPy was used to perform addition, subtraction, multiplication, division, and statistical calculations such as mean, median, minimum, maximum, standard deviation, and sum.

Python Source Code

```
import numpy as np

#Mathematical operations:
```

```
week1 = np.array([100, 200, 300, 400, 500])

week2 = np.array([50, 100, 150, 200, 250])


print("Week 1 sales data:")

print(week1)

print("\nWeek 2 sales data:")

print(week2)


addition = np.add(week1, week2)

print("\nAddition:")

print(addition)


subtraction = np.subtract(week1, week2)

print("\nSubtraction:")

print(subtraction)


multiplication = np.multiply(week1, week2)

print("\nMultiplication:")

print(multiplication)


division = np.divide(week1, week2)

print("\nDivision:")

print(division)


#Statistical Operations:

sales = np.array([120, 250, 180, 300, 150, 400, 220, 350, 280, 200])
```

```
print("\nSales data:")

print(sales)

print("\nMean:", np.mean(sales))

print("Median:", np.median(sales))

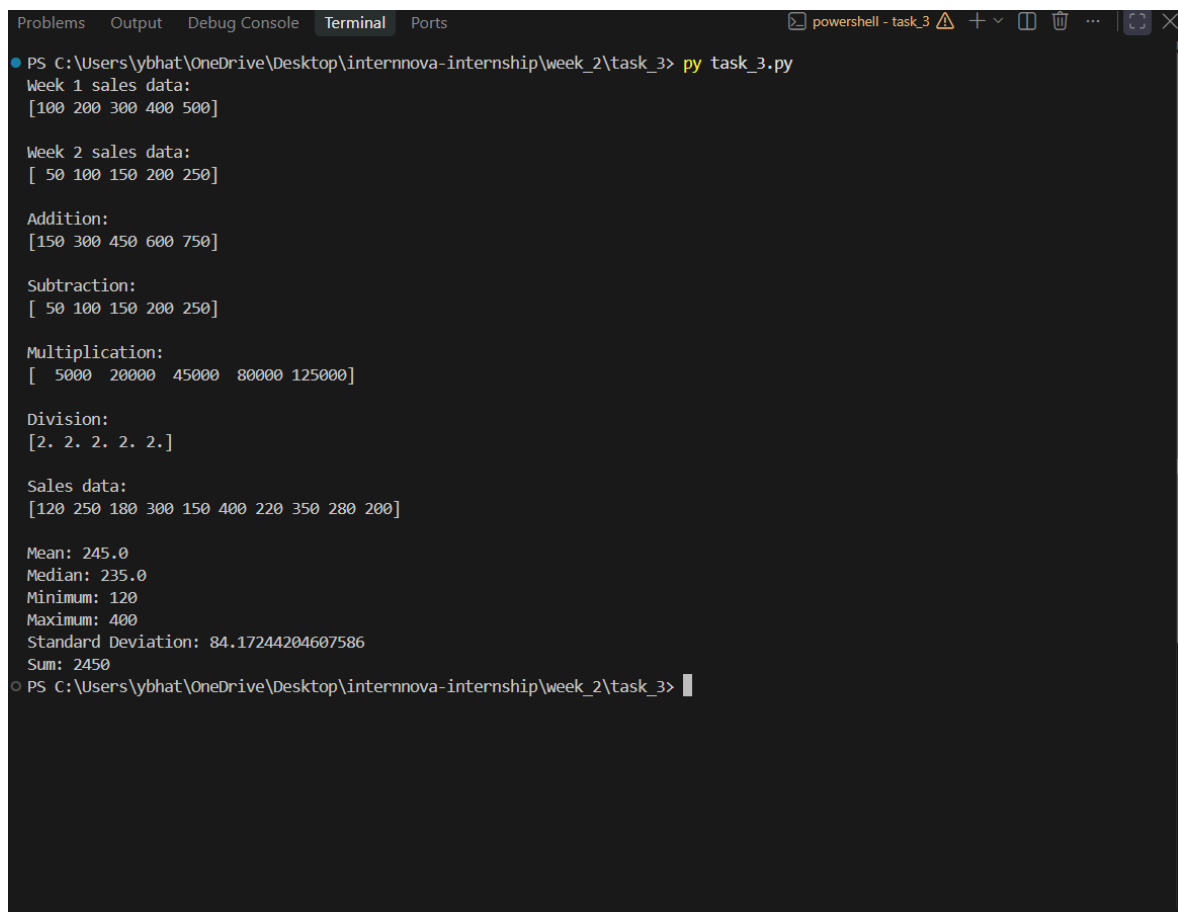
print("Minimum:", np.min(sales))

print("Maximum:", np.max(sales))

print("Standard Deviation:", np.std(sales))

print("Sum:", np.sum(sales))
```

Screenshot of Program Output



```
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_3> py task_3.py
Week 1 sales data:
[100 200 300 400 500]

Week 2 sales data:
[ 50 100 150 200 250]

Addition:
[150 300 450 600 750]

Subtraction:
[ 50 100 150 200 250]

Multiplication:
[ 5000 20000 45000 80000 125000]

Division:
[2.  2.  2.  2.  2.]

Sales data:
[120 250 180 300 150 400 220 350 280 200]

Mean: 245.0
Median: 235.0
Minimum: 120
Maximum: 400
Standard Deviation: 84.17244204607586
Sum: 2450
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_3>
```

4. Task 4 – Pandas Series & DataFrame

Objective

To understand Pandas Series and DataFrames.

Brief Explanation

A Pandas Series and DataFrame were created and inspected. Column names and indexes were displayed, and a new column was added to the DataFrame.

Python Source Code

```
import pandas as pd

# Pandas Series

marks = pd.Series([85, 90, 78, 92, 88])

print("Pandas Series:")
print(marks)

# Student Information

students = {

    "Name": ["Rahul", "Priya", "Amit", "Neha", "Rohan"],

    "Age": [20, 21, 20, 22, 21],

    "Marks": [85, 90, 78, 92, 88],

    "Department": ["Computer", "IT", "Computer", "ECE", "IT"]

}
```

```
# Create DataFrame

df = pd.DataFrame(students)


# Display DataFrame

print("\nStudent DataFrame:")

print(df)


# Display column names

print("\nColumn Names:")

print(df.columns)


# Display DataFrame index

print("\nDataFrame Index:")

print(df.index)


# Add a new column

df["Result"] = ["Pass", "Pass", "Pass", "Pass", "Pass"]
```

```
# Display updated DataFrame

print("\nUpdated DataFrame:")

print(df)
```

Screenshot of Program Output

```
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship> cd week_2/task_4
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_4> py task_4.py

Pandas Series:
0    85
1    90
2    78
3    92
4    88
dtype: int64

Student DataFrame:
   Name  Age  Marks Department
0  Rahul   20    85   Computer
1  Priya   21    90         IT
2   Amit   20    78   Computer
3  Neha   22    92         ECE
4  Rohan   21    88         IT

Column Names:
Index(['Name', 'Age', 'Marks', 'Department'], dtype='str')

DataFrame Index:
RangeIndex(start=0, stop=5, step=1)

Updated DataFrame:
   Name  Age  Marks Department Result
0  Rahul   20    85   Computer   Pass
1  Priya   21    90         IT   Pass
2   Amit   20    78   Computer   Pass
3  Neha   22    92         ECE   Pass
4  Rohan   21    88         IT   Pass
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_4>
```

5. Task 5 – Reading & Inspecting Data

Objective

To learn how to load and inspect a CSV dataset using Pandas.

Brief Explanation

A CSV dataset was loaded using Pandas. The first and last records, dataset dimensions, column names, data types, dataset information, and statistical summary were examined.

Python Source Code

```
import pandas as pd

#Read the CSV file

df = pd.read_csv("sales_data.csv")

#Display the first 5 rows

print("First 5 Rows:")

print(df.head())

#Display the last 5 rows

print("\nLast 5 Rows:")

print(df.tail())

#Check the number of rows and columns

print("\nNumber of Rows and Columns:")

print(df.shape)

print("Number of Rows:", df.shape[0])

print("Number of Columns:", df.shape[1])

#Display column names

print("\nColumn Names:")
```

```

print(df.columns)

#Check data types

print("\nData Types:")

print(df.dtypes)

#Display DataFrame information

print("\nDataFrame Information:")

df.info()

#Display statistical summary

print("\nStatistical Summary:")

print(df.describe())

```

Screenshot of Program Output

```

PS C:\Users\ybhath\OneDrive\Desktop\internnova-internship\week_2\task_5> py task_5.py
First 5 Rows:
  Order_ID  Product  Category  Quantity  Price  Sales
0    1001   Laptop  Electronics      2    50000  100000
1    1002    Mouse  Electronics      5      800    4000
2    1003  Keyboard  Electronics      3    1500    4500
3    1004    Chair  Furniture      4     5000   20000
4    1005     Desk  Furniture      2    10000   20000

Last 5 Rows:
  Order_ID  Product  Category  Quantity  Price  Sales
5    1006  Monitor  Electronics      3   15000  45000
6    1007  Notebook  Stationery     10     100    1000
7    1008     Pen  Stationery     20      50    1000
8    1009  Printer  Electronics      2   12000  24000
9    1010    Table  Furniture      3     7000  21000

Number of Rows and Columns:
(10, 6)
Number of Rows: 10
Number of Columns: 6

Column Names:
Index(['Order_ID', 'Product', 'Category', 'Quantity', 'Price', 'Sales'], dtype='str')

Data Types:
Order_ID    int64
Product      str
Category     str
Quantity    int64
Price       int64
Sales       int64
dtype: object

```

```
Problems Output Debug Console Terminal Ports powershell - task_5 + - [X] X
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_5> py task_5.py

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Order_ID    10 non-null     int64
1   Product     10 non-null     str
2   Category    10 non-null     str
3   Quantity    10 non-null     int64
4   Price       10 non-null     int64
5   Sales       10 non-null     int64
dtypes: int64(4), str(2)
memory usage: 772.0 bytes

Statistical Summary:
      Order_ID  Quantity      Price      Sales
count      10.00000  10.000000    10.000000    10.000000
mean    1005.50000   5.400000  10145.000000  24050.000000
std         3.02765   5.660781  14969.198932  29993.008445
min     1001.00000   2.000000   50.000000   1000.000000
25%     1003.25000   2.250000   975.000000   4125.000000
50%     1005.50000   3.000000   6000.000000  20000.000000
75%     1007.75000   4.750000  11500.000000  23250.000000
max     1010.00000  20.000000  50000.000000 100000.000000
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_5> |
```

6. Task 6 – Selecting, Filtering & Sorting Data

Objective

To select, filter, and sort records using Pandas.

Brief Explanation

Specific columns and rows were selected from the DataFrame. Conditions were used to filter records, including multiple conditions. The data was also sorted in ascending and descending order.

Python Source Code

```
import pandas as pd
```

```
#Read the dataset

df = pd.read_csv("sales_data.csv")


print("Original Data:")

print(df)

#Selecting Specific Columns

print("\n1. Selected Columns - Product and Sales:")

selected_columns = df[["Product", "Sales"]]

print(selected_columns)

#Selecting Specific Rows

print("\n2. Selected Rows - First 3 Rows:")

selected_rows = df.iloc[0:3]

print(selected_rows)

#Filtering Using One Condition

print("\n3. Products with Sales Greater Than 20000:")

filtered_data = df[df["Sales"] > 20000]

print(filtered_data)


#Filtering Using Multiple Conditions

print("\n4. Electronics Products with Sales Greater Than 10000:")

multiple_filter = df[

    (df["Category"] == "Electronics") &

    (df["Sales"] > 10000)
```

```

]

print(multiple_filter)

#Sorting in Ascending Order

print("\n5. Data Sorted by Sales - Ascending:")

ascending_data = df.sort_values(by="Sales", ascending=True)

print(ascending_data)

#Sorting in Descending Order

print("\n6. Data Sorted by Sales - Descending:")

descending_data = df.sort_values(by="Sales", ascending=False)

print(descending_data)

```

Screenshot of Program Output

```

PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_6> py task_6.py
Original Data:

```

	Order_ID	Product	Category	Quantity	Price	Sales
0	1001	Laptop	Electronics	2	50000	100000
1	1002	Mouse	Electronics	5	800	4000
2	1003	Keyboard	Electronics	3	1500	4500
3	1004	Chair	Furniture	4	5000	20000
4	1005	Desk	Furniture	2	10000	20000
5	1006	Monitor	Electronics	3	15000	45000
6	1007	Notebook	Stationery	10	100	1000
7	1008	Pen	Stationery	20	50	1000
8	1009	Printer	Electronics	2	12000	24000
9	1010	Table	Furniture	3	7000	21000

```

1. Selected Columns - Product and Sales:

```

	Product	Sales
0	Laptop	100000
1	Mouse	4000
2	Keyboard	4500
3	Chair	20000
4	Desk	20000
5	Monitor	45000
6	Notebook	1000
7	Pen	1000
8	Printer	24000
9	Table	21000

```

2. Selected Rows - First 3 Rows:

```

	Order_ID	Product	Category	Quantity	Price	Sales
0	1001	Laptop	Electronics	2	50000	100000
1	1002	Mouse	Electronics	5	800	4000
2	1003	Keyboard	Electronics	3	1500	4500

```
PS C:\Users\yibhat\OneDrive\Desktop\internnova-internship\week_2\task_6> py task_6.py
3. Products with Sales Greater Than 20000:
  Order_ID  Product  Category  Quantity  Price  Sales
0      1001  Laptop  Electronics      2   50000  100000
5      1006  Monitor  Electronics      3   15000   45000
8      1009  Printer  Electronics      2   12000   24000
9      1010   Table   Furniture      3    7000   21000

4. Electronics Products with Sales Greater Than 10000:
  Order_ID  Product  Category  Quantity  Price  Sales
0      1001  Laptop  Electronics      2   50000  100000
5      1006  Monitor  Electronics      3   15000   45000
8      1009  Printer  Electronics      2   12000   24000

5. Data Sorted by Sales - Ascending:
  Order_ID  Product  Category  Quantity  Price  Sales
6      1007  Notebook  Stationery     10     100    1000
7      1008     Pen   Stationery     20      50    1000
1      1002   Mouse  Electronics      5     800    4000
2      1003  Keyboard  Electronics      3     1500   4500
4      1005   Desk   Furniture      2   10000  20000
3      1004   Chair   Furniture      4     5000  20000
9      1010   Table   Furniture      3     7000  21000
8      1009  Printer  Electronics      2   12000  24000
5      1006  Monitor  Electronics      3   15000  45000
0      1001  Laptop  Electronics      2   50000  100000

6. Data Sorted by Sales - Descending:
  Order_ID  Product  Category  Quantity  Price  Sales
0      1001  Laptop  Electronics      2   50000  100000
5      1006  Monitor  Electronics      3   15000   45000
8      1009  Printer  Electronics      2   12000   24000
9      1010   Table   Furniture      3    7000   21000
4      1005   Desk   Furniture      2   10000  20000
3      1004   Chair   Furniture      4     5000  20000
2      1003  Keyboard  Electronics      3     1500   4500
1      1002   Mouse  Electronics      5     800    4000
7      1008     Pen   Stationery     20      50    1000
6      1007  Notebook  Stationery     10     100    1000
```

7. Task 7 – Handling Missing Values

Objective

To identify and handle missing values in a dataset.

Brief Explanation

Missing values were identified and counted using Pandas. Missing records were removed or filled using appropriate methods. The dataset was compared before and after handling missing values.

Python Source Code

```
import pandas as pd

# Read the CSV file
df = pd.read_csv("task7_missing_values.csv")
```

```
print("Original Data:")
print(df)

# 1. Detect missing values
print("\n1. Missing Values:")
print(df.isnull())

# 2. Count missing values
print("\n2. Missing Values Count:")
print(df.isnull().sum())

# 3. Remove rows containing missing values
print("\n3. Data After Removing Missing Values:")
df_dropped = df.dropna()
print(df_dropped)

# 4. Fill missing numeric values with the mean
df_filled = df.copy()

df_filled["Age"] = df_filled["Age"].fillna(df_filled["Age"].mean())
df_filled["Marks"] =
df_filled["Marks"].fillna(df_filled["Marks"].mean())

# Fill missing categorical value
df_filled["Department"] = df_filled["Department"].fillna("Unknown")

print("\n4. Data After Filling Missing Values:")
print(df_filled)

# 5. Verify missing values
print("\n5. Missing Values After Filling:")
print(df_filled.isnull().sum())
```

Screenshot of Program Output

```
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_7> py task_7.py

1. Missing Values:
   Name  Age  Marks  Department
0  False  False  False      False
1  False  False   True      False
2  False   True  False      False
3  False  False  False       True
4  False  False   True      False

2. Missing Values Count:
   Name  Age  Marks  Department
0      0     1     2          1
dtype: int64

3. Data After Removing Missing Values:
   Name  Age  Marks  Department
0  Rahul  20.0  85.0    Computer

4. Data After Filling Missing Values:
   Name  Age  Marks  Department
0  Rahul  20.0  85.0    Computer
1  Priya  21.0  85.0         IT
2   Amit  21.0  78.0    Computer
3   Neha  22.0  92.0    Unknown
4   Rohan 21.0  85.0         IT

5. Missing Values After Filling:
   Name  Age  Marks  Department
0      0     0     0          0
dtype: int64
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_7>
```

8. Task 8 – Merge, Concatenate, GroupBy & Pivot Table

Objective

To combine and summarize datasets using Pandas.

Brief Explanation

DataFrames were merged and concatenated using common columns. GroupBy operations were performed to calculate aggregate values, and Pivot Tables were created to summarize the data.

Python Source Code

```
import pandas as pd

# Read CSV files

customers = pd.read_csv("task8_customers.csv")

orders = pd.read_csv("task8_orders.csv")

january = pd.read_csv("task8_sales_january.csv")

february = pd.read_csv("task8_sales_february.csv")

print("Customers Data:")

print(customers)

print("\nOrders Data:")

print(orders)

print("\nJanuary Sales:")

print(january)

print("\nFebruary Sales:")

print(february)
```

```
# 1. Concatenate January and February sales

combined_sales = pd.concat(

    [january, february],

    ignore_index=True

)

print("\n1. Combined January and February Sales:")

print(combined_sales)


# 2. Merge customers and orders

merged_data = pd.merge(

    orders,

    customers,

    on="Customer_ID",

    how="inner"

)

print("\n2. Merged Customer and Order Data:")

print(merged_data)


# 3. GroupBy - Total sales by category

category_sales = orders.groupby("Category")["Sales"].sum()
```

```
print("\n3. Total Sales by Category:")

print(category_sales)


# 4. GroupBy with multiple statistics

category_summary = orders.groupby("Category")["Sales"].agg(

    ["sum", "mean", "count"]

)

print("\n4. Category Sales Summary:")

print(category_summary)


# 5. Pivot Table

pivot = pd.pivot_table(

    merged_data,

    values="Sales",

    index="City",

    columns="Category",

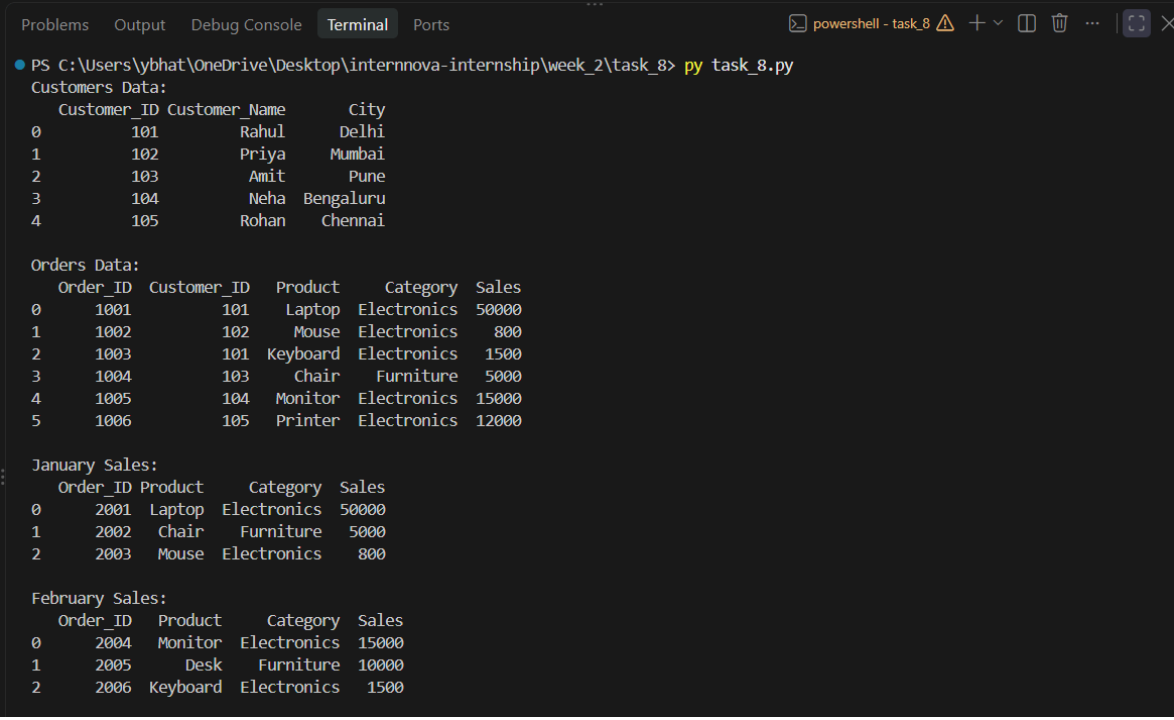
    aggfunc="sum",

    fill_value=0

)
```

```
print("\n5. Pivot Table - Sales by City and Category:")  
  
print(pivot)
```

Screenshot of Program Output



```
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_8> py task_8.py  
Customers Data:  
Customer_ID Customer_Name City  
0 101 Rahul Delhi  
1 102 Priya Mumbai  
2 103 Amit Pune  
3 104 Neha Bengaluru  
4 105 Rohan Chennai  
  
Orders Data:  
Order_ID Customer_ID Product Category Sales  
0 1001 101 Laptop Electronics 50000  
1 1002 102 Mouse Electronics 800  
2 1003 101 Keyboard Electronics 1500  
3 1004 103 Chair Furniture 5000  
4 1005 104 Monitor Electronics 15000  
5 1006 105 Printer Electronics 12000  
  
January Sales:  
Order_ID Product Category Sales  
0 2001 Laptop Electronics 50000  
1 2002 Chair Furniture 5000  
2 2003 Mouse Electronics 800  
  
February Sales:  
Order_ID Product Category Sales  
0 2004 Monitor Electronics 15000  
1 2005 Desk Furniture 10000  
2 2006 Keyboard Electronics 1500
```

```
Problems Output Debug Console Terminal Ports
powershell - task_8 + - - - | [X] X

PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_8> py task_8.py
1. Combined January and February Sales:
   Order_ID Product Category Sales
0    2001  Laptop Electronics 50000
1    2002   Chair Furniture  5000
2    2003  Mouse Electronics  800
3    2004 Monitor Electronics 15000
4    2005   Desk Furniture 10000
5    2006 Keyboard Electronics  1500

2. Merged Customer and Order Data:
   Order_ID Customer_ID Product Category Sales Customer_Name City
0    1001         101  Laptop Electronics 50000      Rahul  Delhi
1    1002         102   Mouse Electronics  800     Priya  Mumbai
2    1003         101 Keyboard Electronics  1500     Rahul  Delhi
3    1004         103   Chair Furniture  5000     Amit   Pune
4    1005         104 Monitor Electronics 15000     Neha  Bengaluru
5    1006         105  Printer Electronics 12000     Rohan  Chennai

3. Total Sales by Category:
Category
Electronics    79300
Furniture       5000
Name: Sales, dtype: int64

4. Category Sales Summary:
      sum  mean  count
Category
Electronics  79300  15860.0    5
Furniture    5000   5000.0    1

5. Pivot Table - Sales by City and Category:
Category Electronics Furniture
City
Bengaluru    15000         0
Chennai      12000         0
Delhi         51500         0
Mumbai        800         0
Pune          0         5000
```

9. Task 9 – Exporting Data

Objective

To export processed data into a CSV file.

Brief Explanation

The processed DataFrame was exported to a CSV file using Pandas. The exported file was then verified to ensure that the processed data was saved correctly.

Python Source Code

```
import pandas as pd

# Read the monthly sales files
```

```
january = pd.read_csv("../task_8/task8_sales_january.csv")

february = pd.read_csv("../task_8/task8_sales_february.csv")

# Combine January and February data

combined_sales = pd.concat(

    [january, february],

    ignore_index=True

)

print("Combined Sales Data:")

print(combined_sales)


# 1. Export DataFrame to CSV

combined_sales.to_csv(

    "task9_combined_sales.csv",

    index=False

)

print("\n1. CSV file exported successfully.")


# 2. Export DataFrame to Excel

combined_sales.to_excel(
```

```

        "task9_combined_sales.xlsx",

        index=False

    )

print("2. Excel file exported successfully.")

# 3. Verify exported data

csv_data = pd.read_csv("task9_combined_sales.csv")

print("\n3. Data Read Back from Exported CSV:")

print(csv_data)

```

Screenshot of Program Output

```

PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_9> py task_9.py
Combined Sales Data:
  Order_ID  Product  Category  Sales
0     2001   Laptop  Electronics  50000
1     2002    Chair   Furniture   5000
2     2003    Mouse  Electronics    800
3     2004   Monitor  Electronics  15000
4     2005     Desk   Furniture  10000
5     2006  Keyboard  Electronics   1500

1. CSV file exported successfully.
2. Excel file exported successfully.

3. Data Read Back from Exported CSV:
  Order_ID  Product  Category  Sales
0     2001   Laptop  Electronics  50000
1     2002    Chair   Furniture   5000
2     2003    Mouse  Electronics    800
3     2004   Monitor  Electronics  15000
4     2005     Desk   Furniture  10000
5     2006  Keyboard  Electronics   1500
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_9>

```

10. Task 10 – Mini Data Analysis Project

Sales Data Analysis Using NumPy & Pandas

Objective

To perform a complete data analysis workflow on a large and messy sales dataset using NumPy and Pandas.

Brief Explanation

A sales dataset containing 15,075 records and 18 columns was analyzed. The dataset contained missing values, duplicate records, inconsistent text values, and invalid numerical values.

The dataset was inspected, cleaned, analyzed, and summarized to identify useful business insights.

Python Source Code

```
import pandas as pd

import numpy as np

df = pd.read_csv("sales_dataset.csv")

# Data Inspection

print("DATASET SHAPE:")

print(df.shape)

print("\nFIRST 5 ROWS:")

print(df.head())

print("\nLAST 5 ROWS:")

print(df.tail())

print("\nCOLUMN NAMES:")

print(df.columns.tolist())

print("\nDATA TYPES:")
```

```
print(df.dtypes)

print("\nDATASET INFO:")

df.info()

print("\nSTATISTICAL SUMMARY:")

print(df.describe())

print("\nMISSING VALUES:")

print(df.isnull().sum())

print("\nDUPLICATE ROWS:")

print(df.duplicated().sum())

# DATA CLEANING

print("\nDATA CLEANING:")

# Create a copy of the original dataset
cleaned_df = df.copy()

# 1. Remove duplicate rows

print("\nDuplicate rows before cleaning:")

print(cleaned_df.duplicated().sum())

cleaned_df = cleaned_df.drop_duplicates()

print("Duplicate rows after cleaning:")

print(cleaned_df.duplicated().sum())

# 2. Convert Order_Date to datetime
cleaned_df["Order_Date"] = pd.to_datetime(

    cleaned_df["Order_Date"],

    errors="coerce"

)
```

```
print("\nOrder_Date data type:")

print(cleaned_df["Order_Date"].dtype)

# 3. Clean text columns

text_columns = [

    "Customer_Name",

    "Product",

    "Category",

    "Subcategory",

    "City",

    "Region",

    "Customer_Segment",

    "Payment_Method",

    "Order_Status",

    "Sales_Channel"

]

for column in text_columns:

    cleaned_df[column] = cleaned_df[column].str.strip()

# Standardize category names

cleaned_df["Category"] = cleaned_df["Category"].str.title()

# Standardize city names

cleaned_df["City"] = cleaned_df["City"].str.title()

# Correct Bangalore variation

cleaned_df["City"] = cleaned_df["City"].replace(

    {"Bangalore": "Bengaluru"}
```

```
)

# 4. Handle missing values

print("\nMissing values before filling:")

print(cleaned_df.isnull().sum()[cleaned_df.isnull().sum() > 0])

# Fill missing customer names

cleaned_df["Customer_Name"] = cleaned_df["Customer_Name"].fillna(

    "Unknown Customer"

)

# Fill missing city

cleaned_df["City"] = cleaned_df["City"].fillna(

    "Unknown"

)

# Fill missing payment method

cleaned_df["Payment_Method"] = cleaned_df["Payment_Method"].fillna(

    "Unknown"

)

# Fill missing discount with median

cleaned_df["Discount"] = cleaned_df["Discount"].fillna(

    cleaned_df["Discount"].median()

)

# Fill missing profit with median

cleaned_df["Profit"] = cleaned_df["Profit"].fillna(

    cleaned_df["Profit"].median()

)
```

```

# 5. Handle invalid Quantity values

print("\nInvalid Quantity values:")

print((cleaned_df["Quantity"] <= 0).sum())

# Replace invalid quantities with NaN

cleaned_df.loc[

    cleaned_df["Quantity"] <= 0,

    "Quantity"

] = np.nan

# Fill invalid quantity values with median

cleaned_df["Quantity"] = cleaned_df["Quantity"].fillna(

    cleaned_df["Quantity"].median()

)

# Convert quantity back to integer

cleaned_df["Quantity"] = cleaned_df["Quantity"].astype(int)

# 6. Handle invalid Discount values

print("\nInvalid Discount values:")

print(

    (

        (cleaned_df["Discount"] < 0) |

        (cleaned_df["Discount"] > 0.50)

    ).sum()

)

# Replace unrealistic discounts with median

median_discount = cleaned_df["Discount"].median()

```

```

cleaned_df.loc[

    (cleaned_df["Discount"] < 0) |

    (cleaned_df["Discount"] > 0.50),

    "Discount"

] = median_discount

# 7. Final missing-value check

print("\nMissing values after cleaning:")

print(cleaned_df.isnull().sum())

# 8. Final dataset information

print("\nCLEANED DATASET")

print("Rows:", len(cleaned_df))

print("Columns:", len(cleaned_df.columns))

print("\nFirst 5 cleaned rows:")

print(cleaned_df.head())

print("\nData types after cleaning:")

print(cleaned_df.dtypes)

# FILTERING AND SORTING

print("\nFILTERING AND SORTING:")

# 1. Select specific columns

print("\nSelected columns:")

print(

    cleaned_df[

        ["Order_ID", "Product", "Category", "Sales", "Profit"]

    ].head(10)

```

```

)

# 2. Select specific rows

print("\nFirst 10 rows:")

print(cleaned_df.iloc[:10])

# 3. Filter high-value sales

high_sales = cleaned_df[cleaned_df["Sales"] > 50000]

print("\nOrders with Sales greater than 50,000:")

print(high_sales[

    ["Order_ID", "Product", "Sales", "Profit"]

].head(10))

# 4. Multiple filtering conditions

high_profit_electronics = cleaned_df[

    (cleaned_df["Category"] == "Electronics") &

    (cleaned_df["Profit"] > 5000)

]

print("\nElectronics orders with Profit greater than 5,000:")

print(

    high_profit_electronics[

        ["Product", "Category", "Sales", "Profit"]

    ].head(10)

)

# 5. Sort by Sales - Descending

top_sales = cleaned_df.sort_values(

    by="Sales",

```

```

        ascending=False
    )

print("\nTop 10 orders by Sales:")

print(

    top_sales[

        ["Order_ID", "Product", "Sales", "Profit"]

    ].head(10)

)

# 6. Sort by Profit - Ascending

lowest_profit = cleaned_df.sort_values(

    by="Profit",

    ascending=True

)

print("\n10 orders with lowest Profit:")

print(

    lowest_profit[

        ["Order_ID", "Product", "Sales", "Profit"]

    ].head(10)

)

# NUMPY ANALYSIS

print("\nNUMPY ANALYSIS:")

sales_array = cleaned_df["Sales"].to_numpy()

profit_array = cleaned_df["Profit"].to_numpy()

quantity_array = cleaned_df["Quantity"].to_numpy()

```

```
# 1. Sales statistics

print("\nSales Statistics:")

print("Total Sales:", np.sum(sales_array))

print("Average Sales:", np.mean(sales_array))

print("Median Sales:", np.median(sales_array))

print("Minimum Sales:", np.min(sales_array))

print("Maximum Sales:", np.max(sales_array))

print("Standard Deviation:", np.std(sales_array))

# 2. Profit statistics

print("\nProfit Statistics:")

print("Total Profit:", np.sum(profit_array))

print("Average Profit:", np.mean(profit_array))

print("Median Profit:", np.median(profit_array))

print("Minimum Profit:", np.min(profit_array))

print("Maximum Profit:", np.max(profit_array))

print("Standard Deviation:", np.std(profit_array))

# 3. Quantity statistics

print("\nQuantity Statistics:")

print("Total Quantity:", np.sum(quantity_array))

print("Average Quantity:", np.mean(quantity_array))

print("Minimum Quantity:", np.min(quantity_array))

print("Maximum Quantity:", np.max(quantity_array))

# GROUPBY ANALYSIS

print("\nGROUPBY ANALYSIS:")
```

```
# 1. Sales and Profit by Category

category_analysis = (

    cleaned_df

    .groupby("Category")

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Average_Sales=("Sales", "mean"),

        Order_Count=("Order_ID", "count")

    )

    .sort_values("Total_Sales", ascending=False)

)

print("\nSales and Profit by Category:")

print(category_analysis)

# 2. Sales and Profit by Region

region_analysis = (

    cleaned_df

    .groupby("Region")

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Order_Count=("Order_ID", "count")

    )

    .sort_values("Total_Sales", ascending=False)
```

```
)

print("\nSales and Profit by Region:")

print(region_analysis)

# 3. Sales by Customer Segment

segment_analysis = (

    cleaned_df

    .groupby("Customer_Segment")

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Order_Count=("Order_ID", "count")

    )

    .sort_values("Total_Sales", ascending=False)

)

print("\nSales by Customer Segment:")

print(segment_analysis)

# 4. Product Performance

product_analysis = (

    cleaned_df

    .groupby("Product")

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Quantity_Sold=("Quantity", "sum"),
```

```

        Order_Count=("Order_ID", "count")

    )

    .sort_values("Total_Sales", ascending=False)
)

print("\nTop 10 Products by Sales:")

print(product_analysis.head(10))

# 5. Monthly Sales Analysis

cleaned_df["Month"] = cleaned_df["Order_Date"].dt.to_period("M")

monthly_sales = (

    cleaned_df

    .groupby("Month")

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Order_Count=("Order_ID", "count")

    )

    .sort_index()

)

print("\nMonthly Sales:")

print(monthly_sales)

# PIVOT TABLE ANALYSIS

print("\nPIVOT TABLE ANALYSIS:")

# 1. Category vs Region - Sales

category_region_pivot = pd.pivot_table(

```

```

        cleaned_df,

        values="Sales",

        index="Category",

        columns="Region",

        aggfunc="sum",

        fill_value=0
    )

print("\nSales by Category and Region:")

print(category_region_pivot)

# 2. Customer Segment vs Category

segment_category_pivot = pd.pivot_table(

    cleaned_df,

    values="Sales",

    index="Customer_Segment",

    columns="Category",

    aggfunc="sum",

    fill_value=0
)

print("\nSales by Customer Segment and Category:")

print(segment_category_pivot)

# 3. Region vs Order Status

region_status_pivot = pd.pivot_table(

    cleaned_df,

    values="Sales",

```

```

        index="Region",

        columns="Order_Status",

        aggfunc="sum",

        fill_value=0

    )

print("\nSales by Region and Order Status:")

print(region_status_pivot)

# BUSINESS INSIGHTS

print("\nBUSINESS INSIGHTS:")

# 1. Highest Sales Category

highest_category = category_analysis["Total_Sales"].idxmax()

highest_category_sales = category_analysis.loc[

    highest_category, "Total_Sales"

]

print("\n1. Highest Sales Category:")

print(highest_category)

print("Sales:", round(highest_category_sales, 2))

# 2. Highest Profit Category

highest_profit_category = category_analysis["Total_Profit"].idxmax()

highest_category_profit = category_analysis.loc[

    highest_profit_category, "Total_Profit"

]

print("\n2. Highest Profit Category:")

print(highest_profit_category)

```

```
print("Profit:", round(highest_category_profit, 2))

# 3. Highest Sales Region

highest_region = region_analysis["Total_Sales"].idxmax()

highest_region_sales = region_analysis.loc[

    highest_region, "Total_Sales"

]

print("\n3. Highest Sales Region:")

print(highest_region)

print("Sales:", round(highest_region_sales, 2))

# 4. Highest Sales Customer Segment

highest_segment = segment_analysis["Total_Sales"].idxmax()

highest_segment_sales = segment_analysis.loc[

    highest_segment, "Total_Sales"

]

print("\n4. Highest Sales Customer Segment:")

print(highest_segment)

print("Sales:", round(highest_segment_sales, 2))

# 5. Top Product

top_product = product_analysis["Total_Sales"].idxmax()

top_product_sales = product_analysis.loc[

    top_product, "Total_Sales"

]

print("\n5. Top Product by Sales:")

print(top_product)
```

```
print("Sales:", round(top_product_sales, 2))

# 6. Most Profitable Product

top_profit_product = product_analysis["Total_Profit"].idxmax()

top_product_profit = product_analysis.loc[

    top_profit_product, "Total_Profit"

]

print("\n6. Most Profitable Product:")

print(top_profit_product)

print("Profit:", round(top_product_profit, 2))

# 7. Best Sales Month

best_month = monthly_sales["Total_Sales"].idxmax()

best_month_sales = monthly_sales.loc[

    best_month, "Total_Sales"

]

print("\n7. Best Sales Month:")

print(best_month)

print("Sales:", round(best_month_sales, 2))

# 8. Lowest Sales Month

lowest_month = monthly_sales["Total_Sales"].idxmin()

lowest_month_sales = monthly_sales.loc[

    lowest_month, "Total_Sales"

]

print("\n8. Lowest Sales Month:")

print(lowest_month)
```

```
print("Sales:", round(lowest_month_sales, 2))

# 9. Order Status Analysis

status_analysis = (

    cleaned_df

    .groupby("Order_Status")

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Order_Count=("Order_ID", "count")

    )

    .sort_values("Total_Sales", ascending=False)

)

print("\n9. Sales by Order Status:")

print(status_analysis)

# 10. Sales Channel Analysis

channel_analysis = (

    cleaned_df

    .groupby("Sales_Channel")

    .agg(

        Total_Sales=("Sales", "sum"),

        Total_Profit=("Profit", "sum"),

        Order_Count=("Order_ID", "count")

    )

    .sort_values("Total_Sales", ascending=False)
```

```

)

print("\n10. Sales by Sales Channel:")

print(channel_analysis)

# EXPORT CLEANED DATASET

print("\nEXPORTING DATASET:")

# Remove helper column before exporting

final_df = cleaned_df.copy()

# Convert Month back to string if it exists

if "Month" in final_df.columns:

    final_df["Month"] = final_df["Month"].astype(str)

# Export

output_file = "cleaned_sales_dataset.csv"

final_df.to_csv(

    output_file,

    index=False

)

print("\nCleaned dataset exported successfully.")

print("File:", output_file)

# Verify exported file

exported_df = pd.read_csv(output_file)

print("\nExported dataset shape:")

print(exported_df.shape)

print("\nFirst 5 rows of exported dataset:")

print(exported_df.head())

```

```
print("\nExport verification successful.")
```

Data Inspection & Cleaning

The dataset was inspected using Pandas functions such as `head()`, `info()`, `describe()`, `isnull()`, and `duplicated()`.

During cleaning:

- 75 duplicate records were removed.
- Missing values were handled.
- Inconsistent text values were standardized.
- 20 invalid quantity values were corrected.
- 15 invalid discount values were corrected.
- The order date was converted to datetime format.

The final cleaned dataset contained **15,000 rows and 18 columns**.

Screenshot of Data Cleaning Output

```
Problems Output Debug Console Terminal Ports
powershell - task_10

PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py
DATASET SHAPE:
(15075, 18)

FIRST 5 ROWS:
  Order_ID Customer_ID Customer_Name Product Category ... Order_Status Sales_Channel Unit_Price Sales Profit
0 ORD106680 C11245 Vikas Kumar Denim Jeans Fashion ... Completed Store 2200.0 3960.0 813.46
1 ORD103291 C10561 Pooja Jain Wireless Mouse Electronics ... Returned Online 1200.0 4320.0 280.01
2 ORD106559 C11561 Sneha Joshi Bookshelf Furniture ... Completed Store 7000.0 42000.0 4776.43
3 ORD107927 C10018 Karan Shah Running Shoes Fashion ... Returned Online 3500.0 3500.0 985.94
4 ORD105665 C11897 Rahul Singh Study Desk Furniture ... Completed Online 9000.0 8100.0 1785.59

[5 rows x 18 columns]

LAST 5 ROWS:
  Order_ID Customer_ID Customer_Name Product ... Sales_Channel Unit_Price Sales Profit
15070 ORD105192 C10677 Rahul Singh Study Desk ... Store 9000.0 15300.0 2595.80
15071 ORD113419 C11387 Rohan Gupta Water Bottle ... Online 700.0 1330.0 101.43
15072 ORD105391 C10896 Arjun Mehta Air Fryer ... Store 8000.0 22800.0 4262.46
15073 ORD100861 C10706 Priya Verma Running Shoes ... Online 3500.0 2975.0 351.66
15074 ORD107271 C10491 Karan Shah USB-C Hub ... Marketplace 2500.0 10625.0 1500.10

[5 rows x 18 columns]

COLUMN NAMES:
['Order_ID', 'Customer_ID', 'Customer_Name', 'Product', 'Category', 'Subcategory', 'City', 'Region', 'Customer_Segment', 'Order_Date',
'Quantity', 'Discount', 'Payment_Method', 'Order_Status', 'Sales_Channel', 'Unit_Price', 'Sales', 'Profit']
```

```
Problems Output Debug Console Terminal Ports
powershell - task_10

PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

DATA TYPES:
Order_ID str
Customer_ID str
Customer_Name str
Product str
Category str
Subcategory str
City str
Region str
Customer_Segment str
Order_Date str
Quantity int64
Discount float64
Payment_Method str
Order_Status str
Sales_Channel str
Unit_Price float64
Sales float64
Profit float64
dtype: object
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py
DATASET INFO:
<class 'pandas.DataFrame'>
RangeIndex: 15075 entries, 0 to 15074
Data columns (total 18 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Order_ID              15075 non-null  str
1   Customer_ID           15075 non-null  str
2   Customer_Name         14801 non-null  str
3   Product               15075 non-null  str
4   Category              15075 non-null  str
5   Subcategory           15075 non-null  str
6   City                  14894 non-null  str
7   Region                15075 non-null  str
8   Customer_Segment      15075 non-null  str
9   Order_Date            15075 non-null  str
10  Quantity              15075 non-null  int64
11  Discount              14923 non-null  float64
12  Payment_Method        14849 non-null  str
13  Order_Status          15075 non-null  str
14  Sales_Channel         15075 non-null  str
15  Unit_Price            15075 non-null  float64
16  Sales                 15075 non-null  float64
17  Profit                14955 non-null  float64
dtypes: float64(4), int64(1), str(13)
memory usage: 3.8 MB

STATISTICAL SUMMARY:

```

	Quantity	Discount	Unit_Price	Sales	Profit
count	15075.000000	14923.000000	15075.000000	15075.000000	14955.000000
mean	2.623018	0.105542	11411.257048	26830.324710	4171.096505
std	1.447058	0.082031	19032.460208	53064.544321	9269.879896
min	-1.000000	-0.050000	700.000000	490.000000	-1277.670000
25%	1.000000	0.050000	2200.000000	4050.000000	485.295000
50%	2.000000	0.100000	5500.000000	9500.000000	1311.620000
75%	4.000000	0.150000	9000.000000	22800.000000	3591.240000
max	6.000000	0.600000	85000.000000	510000.000000	141295.220000

```
Problems Output Debug Console Terminal Ports
PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py
dtypes: float64(4), int64(1), str(13)
memory usage: 3.8 MB

STATISTICAL SUMMARY:

```

	Quantity	Discount	Unit_Price	Sales	Profit
count	15075.000000	14923.000000	15075.000000	15075.000000	14955.000000
mean	2.623018	0.105542	11411.257048	26830.324710	4171.096505
std	1.447058	0.082031	19032.460208	53064.544321	9269.879896
min	-1.000000	-0.050000	700.000000	490.000000	-1277.670000
25%	1.000000	0.050000	2200.000000	4050.000000	485.295000
50%	2.000000	0.100000	5500.000000	9500.000000	1311.620000
75%	4.000000	0.150000	9000.000000	22800.000000	3591.240000
max	6.000000	0.600000	85000.000000	510000.000000	141295.220000

```
MISSING VALUES:
Order_ID          0
Customer_ID       0
Customer_Name     274
Product           0
Category          0
Subcategory       0
City              181
Region            0
Customer_Segment  0
Order_Date        0
Quantity          0
Discount          152
Payment_Method    226
Order_Status      0
Sales_Channel     0
Unit_Price        0
Sales             0
Profit            120
dtype: int64

DUPLICATE ROWS:
75
PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10>
```

```

PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

DATA CLEANING:

Duplicate rows before cleaning:
75
Duplicate rows after cleaning:
0

Order_Date data type:
datetime64[us]

Missing values before filling:
Customer_Name    270
City             180
Discount         150
Payment_Method   225
Profit           120
dtype: int64

Invalid Quantity values:
20

Invalid Discount values:
15

```

```

PS C:\Users\ybbhat\OneDrive\Desktop\internova-internship\week_2\task_10> py task_10.py

Missing values after cleaning:
Order_ID          0
Customer_ID       0
Customer_Name      0
Product           0
Category          0
Subcategory       0
City              0
Region            0
Customer_Segment  0
Order_Date        0
Quantity          0
Discount          0
Payment_Method     0
Order_Status      0
Sales_Channel     0
Unit_Price        0
Sales             0
Profit            0
dtype: int64

CLEANED DATASET
Rows: 15000
Columns: 18

First 5 cleaned rows:
   Order_ID Customer_ID Customer_Name  Product  Category  ... Order_Status Sales_Channel Unit_Price  Sales  Profit
0  ORD106680  C11245   Vikas Kumar   Denim Jeans  Fashion  ... Completed      Store      2200.0   3960.0   813.46
1  ORD103291  C10561   Pooja Jain  Wireless Mouse  Electronics  ... Returned      Online     1200.0   4320.0   280.01
2  ORD106559  C11561   Sneha Joshi   Bookshelf     Furniture  ... Completed      Store     7000.0  42000.0  4776.43
3  ORD107927  C10018   Karan Shah   Running Shoes  Fashion  ... Returned      Online     3500.0   3500.0   985.94
4  ORD105665  C11897   Rahul Singh   Study Desk     Furniture  ... Completed      Online     9000.0   8100.0  1785.59

[5 rows x 18 columns]

```

powershell - task_10

```
Data types after cleaning:
Order_ID          str
Customer_ID       str
Customer_Name     str
Product           str
Category          str
Subcategory       str
City              str
Region            str
Customer_Segment  str
Order_Date        datetime64[us]
Quantity          int64
Discount          float64
Payment_Method     str
Order_Status      str
Sales_Channel     str
Unit_Price        float64
Sales             float64
Profit            float64
dtype: object
PS C:\Users\ybbhat\OneDrive\Desktop\internnova-internship\week_2\task_10>
```

powershell - task_10

FILTERING AND SORTING:

	Order ID	Product	Category	Sales	Profit
0	ORD106680	Denim Jeans	Fashion	3960.0	813.46
1	ORD103291	Wireless Mouse	Electronics	4320.0	280.01
2	ORD106559	Bookshelf	Furniture	42000.0	4776.43
3	ORD107927	Running Shoes	Fashion	3500.0	985.94
4	ORD105665	Study Desk	Furniture	8100.0	1785.59
5	ORD106062	Running Shoes	Fashion	5950.0	1424.43
6	ORD106614	Denim Jeans	Fashion	4400.0	778.91
7	ORD103703	Smartphone X	Electronics	39900.0	4331.19
8	ORD104915	Smartphone X	Electronics	37800.0	7976.66
9	ORD111336	Office Chair	Furniture	22800.0	2322.13

	Order_ID	Customer_ID	Customer_Name	Product	Category	...	Order_Status	Sales_Channel	Unit_Price	Sales	Profit
0	ORD106680	C11245	Vikas Kumar	Denim Jeans	Fashion	...	Completed	Store	2200.0	3960.0	813.46
1	ORD103291	C10561	Pooja Jain	Wireless Mouse	Electronics	...	Returned	Online	1200.0	4320.0	280.01
2	ORD106559	C11561	Sneha Joshi	Bookshelf	Furniture	...	Completed	Store	7000.0	42000.0	4776.43
3	ORD107927	C10018	Karan Shah	Running Shoes	Fashion	...	Returned	Online	3500.0	3500.0	985.94
4	ORD105665	C11897	Rahul Singh	Study Desk	Furniture	...	Completed	Online	9000.0	8100.0	1785.59
5	ORD106062	C10426	Vikas Kumar	Running Shoes	Fashion	...	Completed	Online	3500.0	5950.0	1424.43
6	ORD106614	C10093	Neha Patel	Denim Jeans	Fashion	...	Completed	Online	2200.0	4400.0	778.91
7	ORD103783	C10508	Sneha Joshi	Smartphone X	Electronics	...	Returned	Marketplace	42000.0	39900.0	4331.19
8	ORD104915	C11437	Karan Shah	Smartphone X	Electronics	...	Completed	Online	42000.0	37800.0	7976.66
9	ORD111336	C10709	Rahul Singh	Office Chair	Furniture	...	Completed	Online	12000.0	22800.0	2322.13

```
[10 rows x 18 columns]
```

```
Problems Output Debug Console Terminal Ports
powershell - task_10

PS C:\Users\ybhathat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

Orders with Sales greater than 50,000:
  Order_ID  Product  Sales  Profit
10  ORD114026  27 Inch Monitor  52800.0  3367.00
16  ORD108328  Smartphone X  252000.0  21463.38
18  ORD114420  Laptop Pro 14  229500.0  20239.71
29  ORD101492  Smartphone X  63000.0  1639.27
61  ORD101088  27 Inch Monitor  88000.0  10510.55
63  ORD112332  Smartphone X  117600.0  14300.59
65  ORD105071  Study Desk  51300.0  14431.67
89  ORD103345  Laptop Pro 14  484500.0  114136.59
95  ORD105206  Smartphone X  79800.0  9481.16
111  ORD107860  Laptop Pro 14  153000.0  27243.72

Electronics orders with Profit greater than 5,000:
  Product  Category  Sales  Profit
8  Smartphone X  Electronics  37800.0  7976.66
16  Smartphone X  Electronics  252000.0  21463.38
18  Laptop Pro 14  Electronics  229500.0  20239.71
61  27 Inch Monitor  Electronics  88000.0  10510.55
63  Smartphone X  Electronics  117600.0  14300.59
71  27 Inch Monitor  Electronics  35200.0  5756.69
89  Laptop Pro 14  Electronics  484500.0  114136.59
95  Smartphone X  Electronics  79800.0  9481.16
108  Smartphone X  Electronics  31500.0  5118.21
111  Laptop Pro 14  Electronics  153000.0  27243.72
```

```
Problems Output Debug Console Terminal Ports
powershell - task_10

PS C:\Users\ybhathat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

Top 10 orders by Sales:
  Order_ID  Product  Sales  Profit
1411  ORD109238  Laptop Pro 14  510000.0  134155.91
12577  ORD106090  Laptop Pro 14  510000.0  141295.22
8276  ORD106859  Laptop Pro 14  510000.0  88998.15
8309  ORD112189  Laptop Pro 14  510000.0  102475.14
3158  ORD109967  Laptop Pro 14  510000.0  102253.26
7947  ORD106058  Laptop Pro 14  510000.0  126570.56
4556  ORD105968  Laptop Pro 14  510000.0  48917.17
12160  ORD109012  Laptop Pro 14  510000.0  135892.04
10908  ORD108860  Laptop Pro 14  510000.0  91036.46
7273  ORD112426  Laptop Pro 14  510000.0  89114.44

10 orders with lowest Profit:
  Order_ID  Product  Sales  Profit
3849  ORD109203  Laptop Pro 14  119000.0  -1277.67
11983  ORD111030  Smartphone X  88200.0  -1051.47
15033  ORD109999  Laptop Pro 14  178500.0  -912.50
13063  ORD114162  Office Chair  25200.0  -562.47
2898  ORD105904  Laptop Pro 14  191250.0  -444.47
14112  ORD106971  Vacuum Cleaner  35625.0  -265.39
3444  ORD109360  Office Chair  36000.0  -253.88
12606  ORD106639  Cookware Set  17500.0  -229.89
2839  ORD114762  27 Inch Monitor  15400.0  -224.64
9035  ORD102077  Wrist Watch  7700.0  -189.71
```

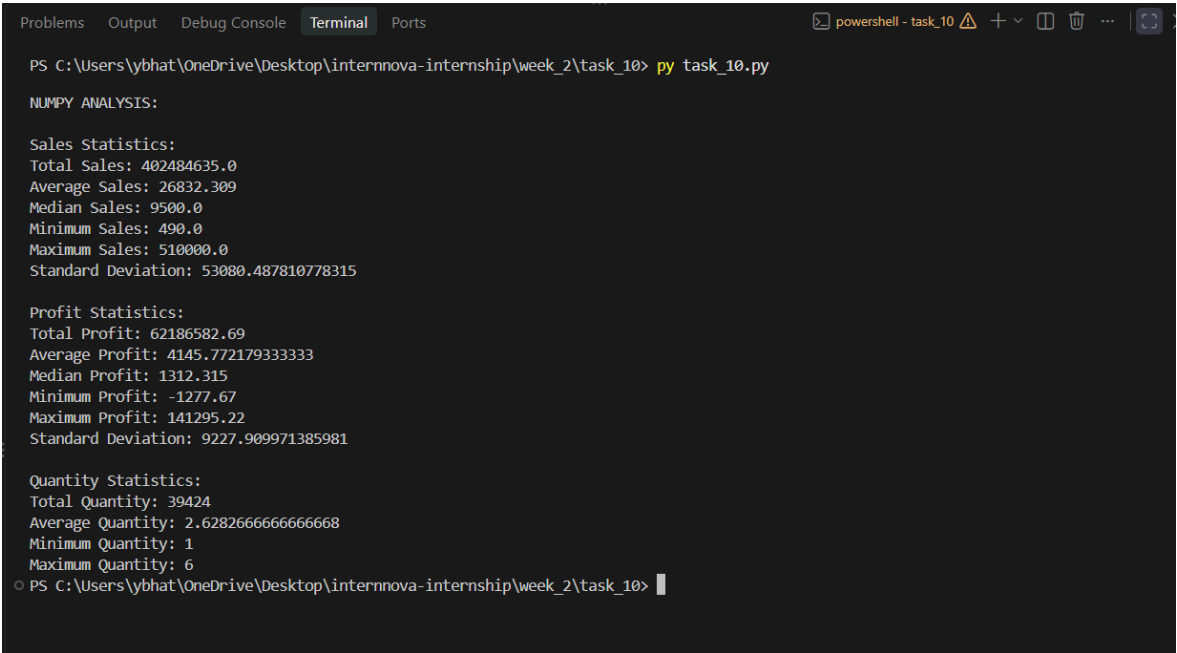
NumPy Analysis

NumPy was used to calculate statistical measures for Sales, Profit, and Quantity.

Key results:

- Total Sales: ₹402,484,635
- Average Sales: ₹26,832.31
- Median Sales: ₹9,500
- Total Profit: ₹62,186,582.69
- Average Profit: ₹4,145.77
- Total Quantity Sold: 39,424

Screenshot of NumPy Analysis



```
Problems Output Debug Console Terminal Ports
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

NUMPY ANALYSIS:

Sales Statistics:
Total Sales: 402484635.0
Average Sales: 26832.309
Median Sales: 9500.0
Minimum Sales: 490.0
Maximum Sales: 510000.0
Standard Deviation: 53080.487810778315

Profit Statistics:
Total Profit: 62186582.69
Average Profit: 4145.772179333333
Median Profit: 1312.315
Minimum Profit: -1277.67
Maximum Profit: 141295.22
Standard Deviation: 9227.909971385981

Quantity Statistics:
Total Quantity: 39424
Average Quantity: 2.6282666666666668
Minimum Quantity: 1
Maximum Quantity: 6
PS C:\Users\ybhat\OneDrive\Desktop\internnova-internship\week_2\task_10> |
```

GroupBy & Pivot Table Analysis

GroupBy analysis was performed based on category, region, customer segment, product, and month.

Pivot Tables were created to analyze:

- Sales by Category and Region
- Sales by Customer Segment and Category
- Sales by Region and Order Status

Screenshot of GroupBy & Pivot Table Output

```
Problems Output Debug Console Terminal Ports
powershell - task_10

PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

GROUPBY ANALYSIS:

Sales and Profit by Category:
      Total_Sales  Total_Profit  Average_Sales  Order_Count
Category
Electronics      271561845.0    41729148.19    61369.908475      4425
Home & Kitchen    54415980.0      8520139.15    14145.042891      3847
Furniture         52018640.0      8093975.36    17627.461877      2951
Fashion          24488170.0      3843319.99    6483.497485      3777

Sales and Profit by Region:
      Total_Sales  Total_Profit  Order_Count
Region
South      82117035.0    1.280454e+07      2974
West       81472045.0    1.260359e+07      3071
East       81168660.0    1.237714e+07      2920
North      80151960.0    1.261520e+07      2999
Central    77574935.0    1.178611e+07      3036

Sales by Customer Segment:
      Total_Sales  Total_Profit  Order_Count
Customer_Segment
Consumer      224607015.0    3.470165e+07      8221
Corporate     101977405.0    1.546258e+07      3803
Small Business  75900215.0    1.202235e+07      2976
```

```
Problems Output Debug Console Terminal Ports
powershell - task_10

PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

Top 10 Products by Sales:
      Total_Sales  Total_Profit  Quantity_Sold  Order_Count
Product
Laptop Pro 14    145490250.0    2.225420e+07      1911      729
Smartphone X     73136700.0    1.108600e+07      1951      735
27 Inch Monitor  38627600.0    6.184975e+06      1961      751
Office Chair     21354000.0    3.311212e+06      1977      743
Vacuum Cleaner   17060100.0    2.675782e+06      2001      749
Air Fryer        15744800.0    2.436579e+06      2191      812
Study Desk       15610500.0    2.482743e+06      1946      743
Bookshelf        11922050.0    1.812714e+06      1915      722
Coffee Maker     11852425.0    1.908751e+06      2025      782
Wrist Watch      10130175.0    1.607118e+06      2064      798

Monthly Sales:
      Total_Sales  Total_Profit  Order_Count
Month
2024-01    17763900.0    2591316.700      621
2024-02    16200250.0    2375427.305      601
2024-03    16228215.0    2539379.585      625
2024-04    15244580.0    2247989.340      582
2024-05    16551610.0    2520021.840      621
2024-06    13990030.0    2109931.425      571
2024-07    16557050.0    2750395.275      644
2024-08    18558270.0    2898497.660      660
2024-09    17476145.0    2596948.140      637
2024-10    18577960.0    2849382.950      636
2024-11    18646445.0    2916645.585      665
2024-12    15857980.0    2482281.325      616
2025-01    17910140.0    3093544.030      718
2025-02    16510105.0    2584784.465      574
2025-03    16845610.0    2701283.615      616
2025-04    16464160.0    2449102.410      627
2025-05    14380750.0    2174018.550      616
2025-06    18010390.0    2740510.930      603
2025-07    17654880.0    2670509.065      611
2025-08    17789660.0    2841745.810      626
2025-09    16033190.0    2447169.830      635
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py
2025-08 17789660.0 2841745.810 626
2025-09 16033190.0 2447169.830 635
2025-10 14370310.0 2218362.275 619
2025-11 16365800.0 2496601.985 614
2025-12 18497205.0 2890732.595 662

PIVOT TABLE ANALYSIS:

Sales by Category and Region:
Region      Central      East      North      South      West
Category
Electronics 50837090.0 55661300.0 53991390.0 56839680.0 54232385.0
Fashion     5070570.0 4705535.0 4960220.0 4662480.0 5089365.0
Furniture   11246110.0 10332700.0 10165730.0 9443830.0 10830270.0
Home & Kitchen 10421165.0 10469125.0 11034620.0 11171045.0 11320025.0

Sales by Customer Segment and Category:
Category    Electronics  Fashion  Furniture  Home & Kitchen
Customer_Segment
Consumer    153875955.0 14000205.0 27575530.0 29155325.0
Corporate   68875350.0 5862225.0 13299070.0 13940760.0
Small Business 48810540.0 4625740.0 11144040.0 11319895.0

Sales by Region and Order Status:
Order_Status Cancelled  Completed  Returned
Region
Central       12944870.0 50815620.0 13814445.0
East          14669630.0 52977445.0 13521585.0
North         12577570.0 54773815.0 12800575.0
South         13445460.0 54711565.0 13960010.0
West          13589010.0 53442260.0 14440775.0
PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10> |
```

Key Findings

- **Electronics** generated the highest category sales at approximately **₹271.56 million**.
- **South** recorded the highest regional sales at approximately **₹82.12 million**.
- **Consumer** was the highest-selling customer segment at approximately **₹224.61 million**.
- **Laptop Pro 14** was the top product by sales at approximately **₹145.49 million**.
- Total sales across the cleaned dataset were approximately **₹402.48 million**.
- Total profit was approximately **₹62.19 million**.

Exported Dataset

The cleaned dataset was exported as:

`cleaned_sales_dataset.csv`

Screenshot of Export Verification

```

PS C:\Users\ybbat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

BUSINESS INSIGHTS:

1. Highest Sales Category:
Electronics
Sales: 271561845.0

2. Highest Profit Category:
Electronics
Profit: 41729148.19

3. Highest Sales Region:
South
Sales: 82117035.0

4. Highest Sales Customer Segment:
Consumer
Sales: 224607015.0

5. Top Product by Sales:
Laptop Pro 14
Sales: 145490250.0

6. Most Profitable Product:
Laptop Pro 14
Profit: 22254195.28

7. Best Sales Month:
2024-11
Sales: 18646445.0

8. Lowest Sales Month:
2024-06
Sales: 13990030.0

```

```

Problems Output Debug Console Terminal Ports
powershell - task_10 + v [ ] [ ] ... [ ] [ ]

PS C:\Users\y\hat\OneDrive\Desktop\internnova-internship\week_2\task_10> py task_10.py

9. Sales by Order Status:
      Total_Sales  Total_Profit  Order_Count
Order_Status
Completed  266720705.0  41402593.80      10002
Returned   68537390.0   10409777.83       2443
Cancelled  67226540.0    10374211.06       2555

10. Sales by Sales Channel:
      Total_Sales  Total_Profit  Order_Count
Sales_Channel
Online  243676585.0   3.733362e+07      8950
Store   80956300.0   1.279319e+07      3055
Marketplace  77851750.0   1.205978e+07      2995

EXPORTING DATASET:

Cleaned dataset exported successfully.
File: cleaned_sales_dataset.csv

Exported dataset shape:
(15000, 19)

First 5 rows of exported dataset:
   Order_ID Customer_ID Customer_Name  Product  Category  ... Sales_Channel Unit_Price  Sales  Profit  Month
0  ORD106680  C11245  Vikas Kumar  Denim Jeans  Fashion  ... Store  2200.0  3960.0  813.46  2024-01
1  ORD103291  C10561  Pooja Jain  Wireless Mouse  Electronics  ... Online  1200.0  4320.0  280.01  2024-11
2  ORD106559  C11561  Sneha Joshi  Bookshelf  Furniture  ... Store  7000.0  42000.0  4776.43  2025-02
3  ORD107927  C10018  Karan Shah  Running Shoes  Fashion  ... Online  3500.0  3500.0  985.94  2025-04
4  ORD105665  C11897  Rahul Singh  Study Desk  Furniture  ... Online  9000.0  8100.0  1785.59  2024-01

[5 rows x 19 columns]

Export verification successful.
PS C:\Users\y\hat\OneDrive\Desktop\internnova-internship\week_2\task_10>

```

Conclusion

Conclusion

This assignment provided practical experience with NumPy and Pandas for data analysis. I learned how to create and manipulate arrays, work with DataFrames, inspect and clean datasets, filter and sort data, perform GroupBy operations, create Pivot Tables, calculate statistics, and export processed data.

The Sales Data Analysis project helped me apply these concepts to a large and messy dataset and extract meaningful business insights from it.

Google Drive Link:

 **week_2_files**